

Привет!

Сравнительный анализ инструментов для управления секретами в **Kubernetes**

Как безопасно хранить секреты в Kubernetes и выбрать подходящее решение для продакшена.



Выполнил:

Васюков Александр



Секреты в Kubernetes

Контекст, в котором возникает задача управления секретами.



Система уровня банка

Десятки микросервисов: платежи, антифрод, мобильное API, скоринг и другие.



Работает в Kubernetes

Kubernetes управляет контейнерами, автоматически масштабирует и обеспечивает отказоустойчивость.



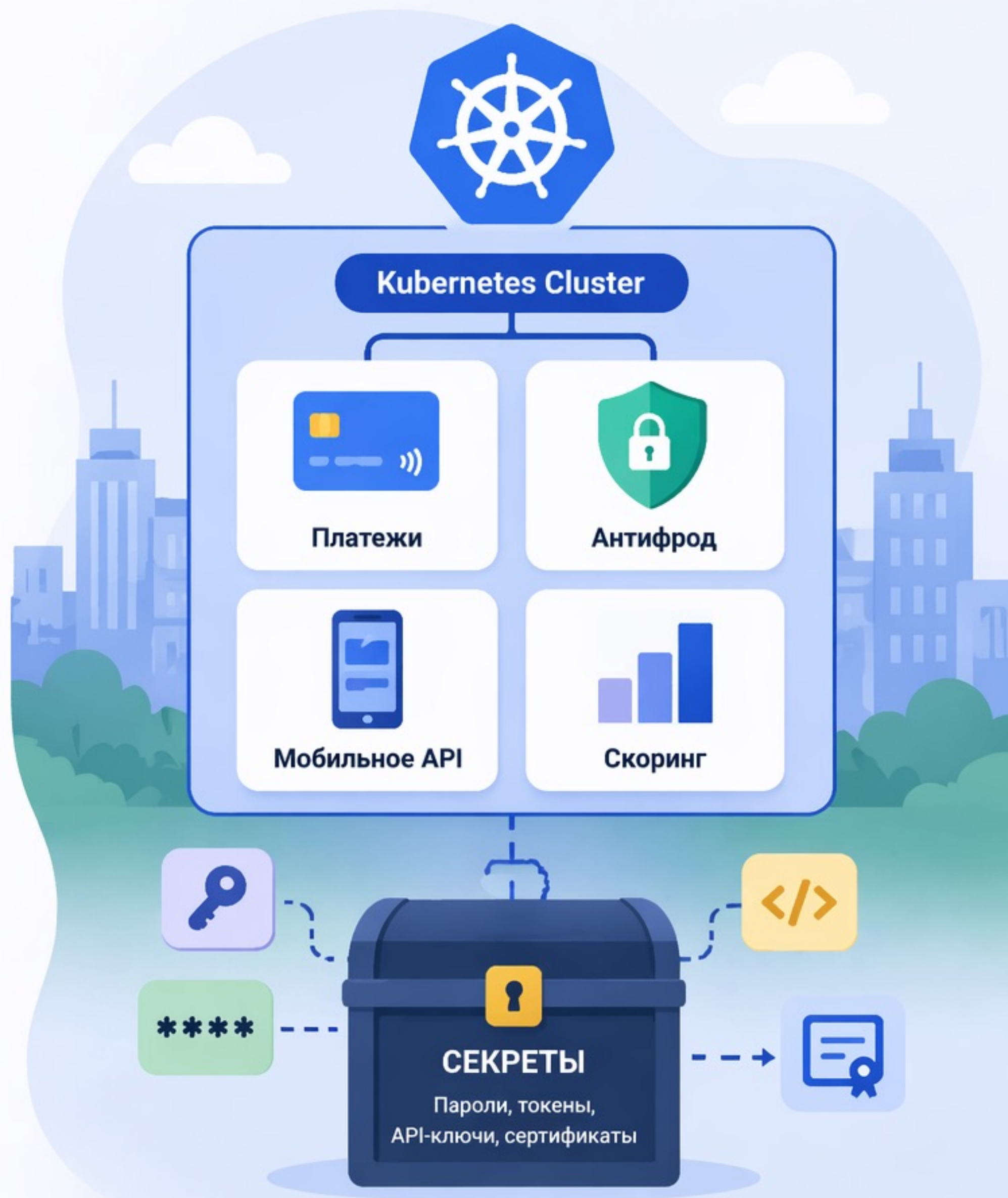
Возникает вопрос

Где хранить пароли, токены, API-ключи и сертификаты так, чтобы они не утекли?



Секреты — это основа безопасности.

От их защиты зависит вся система.



Проблемы с секретами

Kubernetes Secrets решают не все задачи. На практике появляются серьёзные риски.

1 Слабая безопасность

По умолчанию это просто base64, а не шифрование.



Любой с доступом может легко декодировать и получить исходный пароль.

2 Проблемы с GitOps

Секреты в YAML могут попасть в Git и остаться там навсегда.



Даже приватный репозиторий — это риск.

3 Нет жизненного цикла и аудита

Нет ротации, нет истории, нет понимания, кто и когда получил доступ.



Нет ротации

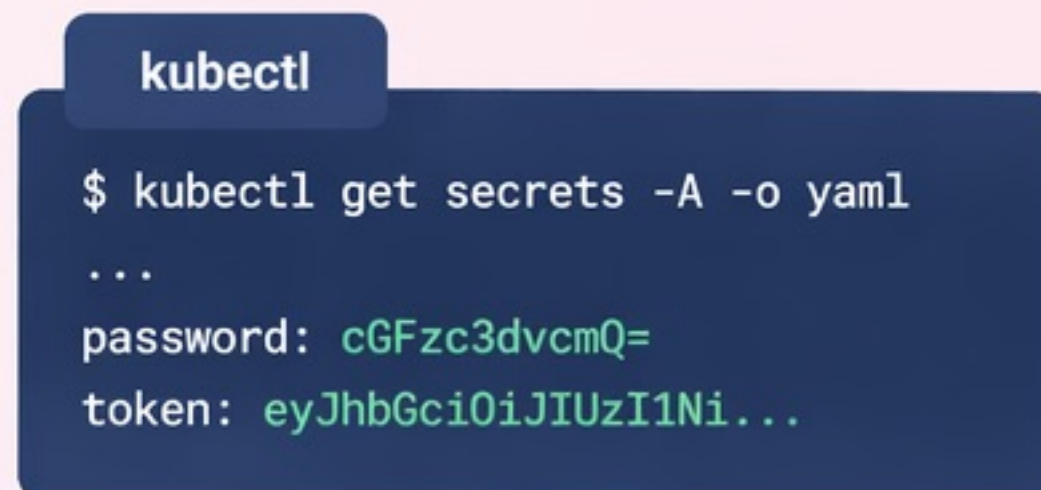


Нет аудита

Реальный сценарий

Разработчик получил доступ к Kubernetes, скачал все Secrets и получил кредиты к базе с платежами.

Это уже потенциальная утечка.



Критерии сравнения

Оцениваем решения с точки зрения требований банковской системы.



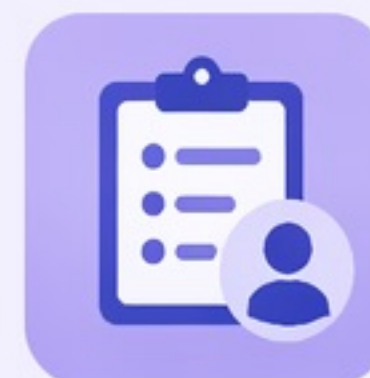
Безопасность

Шифрование, контроль доступа, минимальные права.



Ротация секретов

Автоматическое обновление и поддержка динамических секретов.



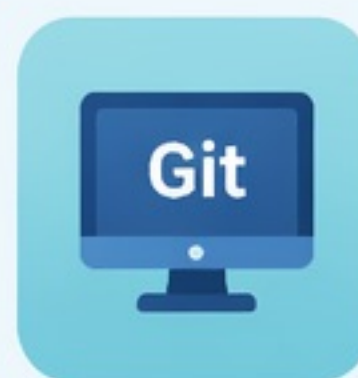
Аудит

Логирование обращений, кто и когда читал секрет.



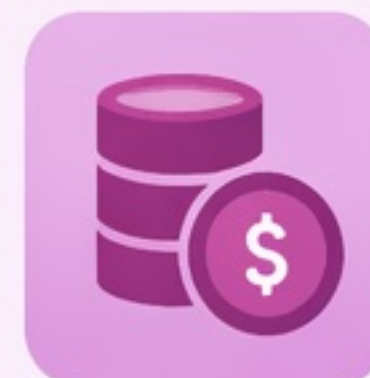
Удобство разработки

Простота использования для команды разработки.



Интеграция с GitOps

Совместимость с Git, CI/CD и инфраструктурой как код.



Стоимость и сложность внедрения

Насколько сложно развернуть, поддерживать и сколько это стоит.



Цель: выбрать решение, которое обеспечивает высокий уровень безопасности, удобно для команды и подходит для продакшена.

Kubernetes Secrets



Встроенный механизм хранения секретов в Kubernetes.
Самый простой способ, но с серьёзными ограничениями.

Как это работает?

Создаём Secret

```
apiVersion: v1
kind: Secret
metadata:
  name: db-credentials
data:
  password: YWJjMTIz
```

Kubernetes хранит его



Хранится в etcd
в base64

Pod использует секрет

Pod



Получает секрет
как переменные
или файлы

Плюсы



Ничего не нужно устанавливать
Уже есть в Kubernetes



Просто использовать
Понятный и быстрый старт



Полная интеграция с Kubernetes
Работает "из коробки"

Минусы



Слабая безопасность
Это не шифрование, а всего лишь base64



Нет ротации
Секреты не обновляются автоматически



Нет аудита
Невозможно отследить, кто читал секрет



Проблемы с GitOps
Нельзя хранить реальные секреты в Git



Вывод:

Подходит только для локальной разработки
и тестовых сервисов. **Для банка — использовать нельзя.**

Sealed Secrets

Инструмент для безопасного хранения секретов в Git.

Секрет шифруется публичным ключом и расшифровывается только внутри кластера.



Как это работает?



Плюсы

- Безопасно для Git**
В репозитории хранится зашифрованный секрет
- Простая установка**
Лёгкий и удобный инструмент
- Работает с GitOps**
Отлично вписывается в CI/CD процессы

Минусы

- Нет централизованного управления**
Каждый кластер — изолированный остров
- Нет динамических секретов**
Только статичные значения
- Нет полноценного аудита**
Непонятно, кто и когда получил доступ

Вывод:

Sealed Secrets — отличный выбор для GitOps и безопасного хранения секретов в Git, но это не полноценная система управления секретами.



HashiCorp Vault и External Secrets Operator

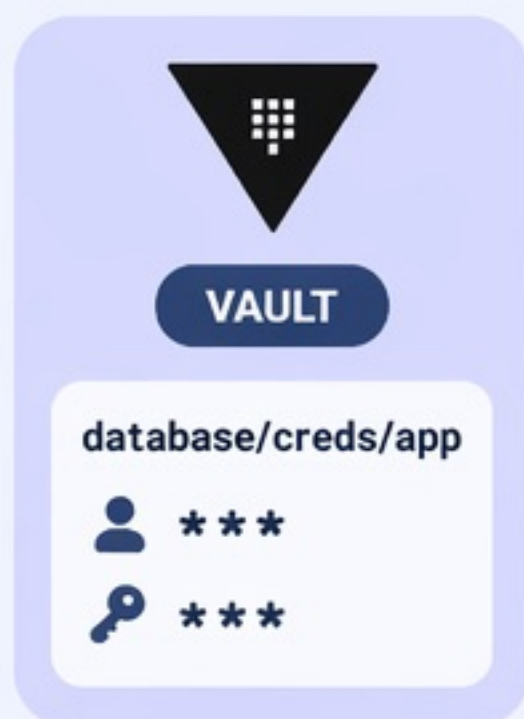


Продвинутое решение: выносит управление секретами из Kubernetes в специализированную систему.

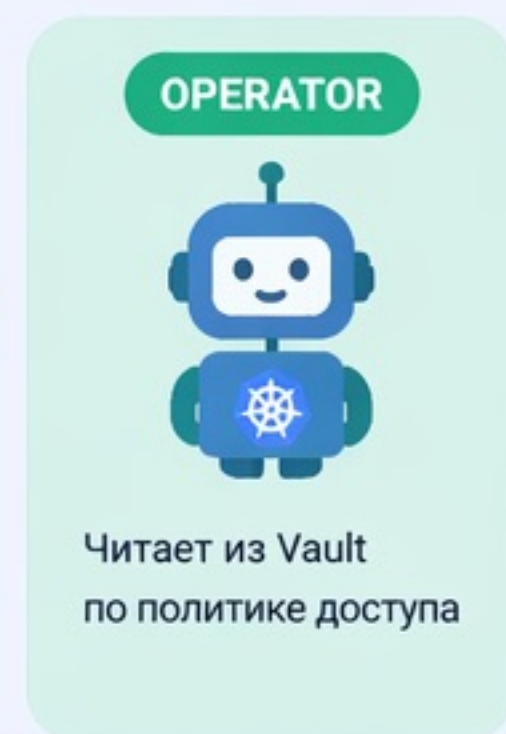
Kubernetes больше не является источником истины для секретов.

Как это работает?

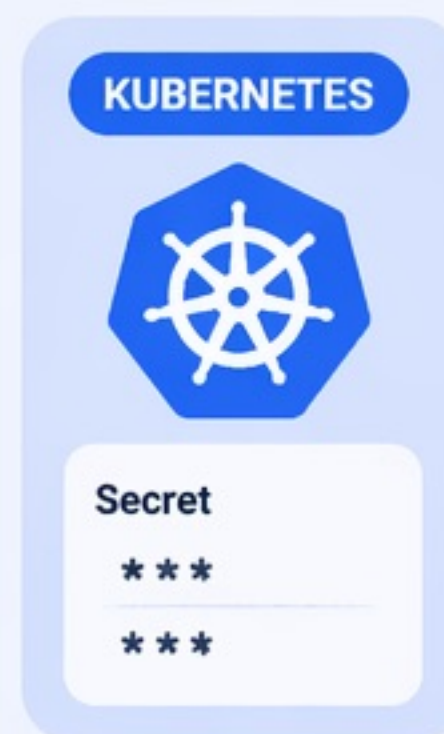
1. Храним секрет в Vault



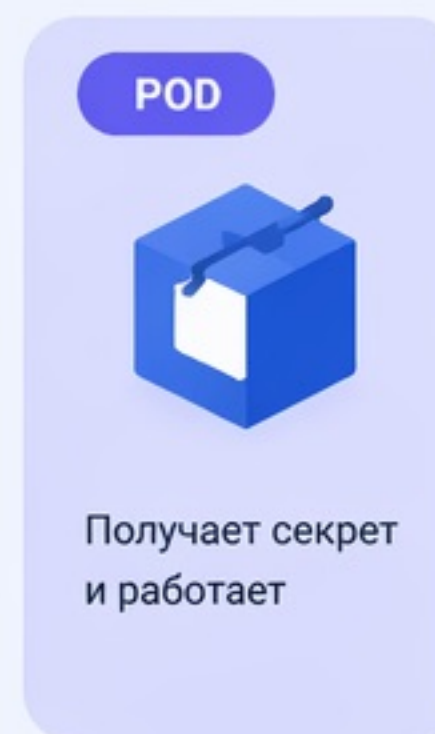
2. External Secrets Operator запрашивает секрет



3. Создаёт Kubernetes Secret



4. Pod использует секрет как обычно



Минусы



Сложность внедрения

Нужно разворачивать, настраивать и поддерживать



Дополнительные задержки

Сетевой запрос к Vault при получении секрета



Инфраструктурные затраты

Требуются ресурсы на Vault и оператор

Плюсы



Высокая безопасность

Шифрование, политики, zero-trust



Ротация и динамические секреты

Автоматическое обновление, временные кредиты



Полноценный аудит

Логирование всех обращений, compliance



Совместимость с GitOps

Секреты не хранятся в Git, CI/CD остаётся неизменным



Прозрачно для разработчика

Работает как обычный Kubernetes Secret

Вывод:

Vault + External Secrets Operator — лучшее решение для продакшена и систем с высокими требованиями к безопасности. Идеально подходит для банков и крупных организаций.



Итоги

Сравним решения и сделаем выводы

Критерий	Kubernetes Secrets	Sealed Secrets	Vault + External Secrets Operator
 Безопасность	 Слабая (base64)	 Хорошая (шифрование для Git)	 Высокая (шифрование, политики, zero-trust)
 Ротация секретов	 Нет	 Нет	 Да, динамические секреты
 Аудит	 Нет	 Ограниченно	 Полный аудит и логирование
 Удобство разработки	 Очень просто	 Просто, работает с Git	 Сложнее внедрить, но прозрачно для разработчика
 Интеграция с GitOps	 Проблемы (секреты в Git)	 Отлично подходит	 Отлично, секреты не хранятся в Git
 Стоимость и сложность внедрения	 Бесплатно, уже в кластере	 Бесплатно, лёгкая установка	 Сложнее, требует инфраструктуры и поддержки



Что выбрать?



Рет-проекты и тесты
Достаточно **Sealed Secrets**



Стартапы и крупные организации
Лучше использовать External Secrets с внешним хранилищем секретов



Системы уровня банка
Практически **нет выбора** — нужен **HashiCorp Vault** или аналогичное решение

Главный вывод

Kubernetes даёт интерфейс для работы с секретами, но не решает задачу их безопасного управления.
В продакшене необходим внешний Secret Manager.

